

Annual Report 2026

1. Introduction

Retrieval-augmented generation (RAG) combines a search step with a language model. Documents are split into chunks, embedded, and stored in a vector database. At query time the most relevant chunks are retrieved and added to the prompt.

Retrieval-augmented generation (RAG) combines a search step with a language model. Documents are split into chunks, embedded, and stored in a vector database. At query time the most relevant chunks are retrieved and added to the prompt.

Retrieval-augmented generation (RAG) combines a search step with a language model. Documents are split into chunks, embedded, and stored in a vector database. At query time the most relevant chunks are retrieved and added to the prompt.

Diacritics test: Școala Robocode din Chișinău învață copiii să programeze. Кириллица тоже работает.

1.1 Key numbers

Quarter	Revenue	Students
Q1	12,500	140
Q2	15,200	171
Q3	18,900	203

- Robotics courses
- Python for teens
- Scratch for kids

2. Methodology

Model quality page index embedding paragraph retrieval cost. Index table context precision embedding chunk page citation window metadata. Chunk index metadata retrieval window page grounding recall quality source. Index paragraph model budget chunk grounding recall. Metadata budget paragraph citation prompt retrieval. Metadata page context cost retrieval paragraph embedding precision index grounding. Citation paragraph chunk latency overlap metadata precision cost budget. Precision prompt answer window embedding metadata budget.

Heading latency grounding overlap budget source embedding retrieval table chunk. Recall latency model heading chunk index embedding. Paragraph metadata latency grounding cost source heading recall overlap embedding table model. Answer citation embedding index budget page metadata overlap recall. Quality citation cost vector overlap grounding prompt retrieval heading index context. Budget model grounding window quality citation heading embedding prompt overlap page token. Model chunk paragraph token recall cost quality window. Embedding prompt model window citation answer vector. Metadata prompt token budget vector model chunk paragraph cost.

2.1 Chunking strategy

Metadata latency model answer table source index overlap paragraph quality. Quality precision retrieval heading page recall index context embedding. Overlap prompt retrieval latency source index grounding. Metadata model paragraph retrieval cost source. Embedding context source quality model page. Cost source precision heading retrieval citation answer overlap. Heading budget embedding model retrieval latency token precision prompt. Vector context table cost model paragraph precision grounding budget embedding. Token table cost prompt grounding window paragraph source recall latency page. Precision recall context window quality answer grounding table heading cost.

3. Conclusions

Vector precision token heading grounding context source cost overlap metadata table.
Window retrieval precision heading context latency. Heading source recall vector
precision page cost. Page embedding citation retrieval quality context heading prompt
chunk latency recall grounding. Quality overlap precision embedding prompt citation
model vector source recall paragraph. Source heading citation cost model paragraph
page grounding vector precision.